JRC BROKER / INTEGRAÇÕES

Guia da API para empresas

Conexões WhatsApp, mensageria, eventos assinados e integração com CCLX / Chatwoot.

Edição técnica 23/10/2026. Contratos extraídos da revisão candidata do Broker. A publicação deste arquivo não certifica implantação no servidor nem homologação de um sistema externo.

O que você recebe

URL da instalação JRC, uma chave exclusiva da organização e documentação OpenAPI. A JRC opera o Broker e o motor de conexão. A empresa integra seu próprio backend e mantém suas telas, agentes, caixas e regras de negócio.

Origem Broker: **https://jrc-broker.jrcws.cloud**
Portal: **/documentation/**
Use a origem fornecida para sua instalação. Chaves reais são entregues separadamente, por canal privado. Este guia não contém credenciais.

Referência do contrato: 0faaa45e31d9bee95485

1. O que a empresa consegue fazer

Área	Disponível pelo perfil documentado
Conexões QR	Descobrir provedores do tenant; criar/listar instâncias; parear, consultar estado, reconectar e desconectar.
Mensageria por chave	Ativar o canal QR; consultar canais, conversas e mensagens; enfileirar texto individual.
Recebimento	Receber eventos em receptor HTTPS com HMAC ou ler eventos por cursor, sem exigir URL Chatwoot previamente.
Controle Chatwoot opcional	Criar/adotar vínculo novo, parear, matricular a primeira identidade quando elegível, confirmar legado/troca, definir agentes e acompanhar operações com credencial vinculada à Account.
Interface dentro do CCLX	Pacote adaptador de QR para incorporação ao host. Instalação exige integração com sessão, permissões e mapeamento do CCLX.

Limites do acesso genérico atual

O envio por chave neste perfil é de **texto individual pelo QR**. Inclui download privado de anexo, mas não envio de mídia, envio em grupos, templates Meta nem administração de Flows. Outras funções do console e integrações diretas usam contratos e permissões próprios. Não assuma que qualquer chave autoriza todas as rotas do Broker.

A integração oficial Meta depende do cadastro, ativos, permissões e onboarding correspondentes. A API de QR não é uma substituição automática do fluxo oficial. O transporte Evolution é interno: o parceiro chama a API JRC, sem receber a chave global do motor.

Critério de funcionamento

HTTP 200/202, build aprovado ou container saudável não comprovam entrega ao WhatsApp. A POC deve demonstrar pareamento, entrada, saída, estados, idempotência, retomada e isolamento entre empresas com números de homologação.

2. Arquitetura para o CCLX

```
Interface CCLX
  -> backend/adaptador CCLX (sessão e permissões locais)
  -> API HTTPS JRC (chave do tenant)
  -> Broker / fila durável / motor QR / WhatsApp

WhatsApp -> Broker -> webhook HMAC ou polling
  -> inbox durável CCLX -> Pub/Sub local
  -> camada de mensageria / caixa / agente no CCLX
```

Informação fornecida pela equipe do CCLX em 22/10: mensageria parcialmente baseada em Chatwoot 3.x; demais módulos em outro banco; integração por APIs e filas Pub/Sub; possível migração futura para 4.x. A versão exata, a tecnologia da fila e os pontos de extensão ainda precisam ser confirmados.

Responsabilidades

JRC	CCLX
Broker, contrato REST, isolamento de tenant, persistência e entrega de eventos.	Guardar a chave no backend e autenticar seus agentes.
Conectar o número e observar estados do transporte.	Mapear instanceId/channelId/conversationId JRC para Account, inbox e conversation locais.
Expor o QR e operações documentadas.	Incorporar a interface QR e respeitar quem pode administrar cada caixa.
Assinar e repetir webhooks dentro dos limites.	Verificar HMAC, persistir eventId único, publicar na fila e evitar loop/duplicação.

Não é necessário compartilhar banco PostgreSQL, Redis, fila interna, SSH ou chave Evolution. O Broker não escreve diretamente no banco do CCLX. Para o caminho genérico, URL/Account/token Chatwoot não são pré-requisitos da emissão da chave.

O pacote QR não instala mensageria ou controller Rails. O CCLX precisa integrar sessão, permissões e cofre, além de construir o consumidor de eventos e o envio pelo compositor local. Os quatro eventos genéricos não atribuem conversas a times/agentes; transferência e retomada de Flow exigem uma ponte de controle própria, homologada no host.

O que falta confirmar com o CCLX

Versão exata 3.x; autenticação e permissões do host; API local de contatos/conversas/mensagens; tecnologia e contrato Pub/Sub; formato dos IDs; receptor HTTPS ou escolha inicial por polling; mapeamento das caixas e dos cinco números da POC.

3. Acesso e configuração inicial

Escopo	Uso
instances:read	Consultar provedores e instâncias.
instances:write	Criar, parear e desconectar instâncias.
messaging:read	Consultar canais, conversas e mensagens.
messaging:send	Ativar canal (também exige instances:write) e enviar texto QR.
webhooks:manage	Cadastrar/revogar receptor, consultar eventos e entregas.

```
GET /v1/provider-accounts?provider=BAILEYS&limit=100
Accept: application/json
X-JRC-API-Key: SUA_CHAVE_PRIVADA
```

A chave identifica a organização. Não envie organizationId no corpo para escolher outro tenant. Os IDs de recursos são UUIDs do Broker, não IDs globais do Evolution. Se a lista de contas de provedor estiver vazia, a JRC precisa provisionar o acesso do tenant antes de criar instâncias.

Validade sugerida para a POC: 15 dias. A chave é revogável e os novos escopos são explícitos. Não guardar a chave em JavaScript público, URL, repositório, print ou pacote de documentação. O Swagger mantém a autorização somente na memória da aba.

Usar o Swagger

Abrir /documentation/, escolher o perfil e clicar Authorize. Informar somente uma credencial por requisição. Escrita fica desabilitada inicialmente; habilitá-la e clicar Execute executa a operação real. O portal não cria conexões ou envia mensagens automaticamente. Há quatro contratos OpenAPI, consultas Postman, POC genérica de 19 operações, catálogo de controle de 15 operações e ambiente vazio. No controle Postman, cada pasta de mutação exige seu próprio allow_*=true e termina o Runner após a ação; Run All não habilita escrita. Preencher IDs e chave somente no ambiente privado. Não publicar respostas ou relatórios brutos.

4. Primeiro número: parear e ativar

Passo 1 - Escolher uma conta e criar instância

```
GET /v1/provider-accounts?provider=BAILEYS&limit=100

POST /v1/instances
Idempotency-Key: poc-instancia-01
Content-Type: application/json

{"name":"POC CCLX", "provider":"BAILEYS",
 "providerAccountId":"UUID_DA_CONTA_RETORNADA"}
```

Guardar response.instance.id como instanceId. Verificar pending, operationId e reconciliationRequired. Não gerar outra intenção de criação ao receber timeout ou resultado incerto.

Passo 2 - Solicitar QR ou reconexão

```
POST /v1/instances/INSTANCE_ID/connect
Idempotency-Key: poc-pareamento-01
Content-Type: application/json

{}

GET /v1/instances/INSTANCE_ID/status
```

Ler action.type. QR_CODE usa action.encoding, value e expiresAt; PAIRING_CODE apresenta code. NONE pode indicar conexão já existente ou operação pendente. Não renderizar qualquer string como URL arbitrária. Reconectar usa o mesmo endpoint /connect, com uma intenção explícita e idempotência própria; não existe rota /reconnect no contrato de instâncias.

Passo 3 - Ativar o canal para mensageria

```
POST /v1/messaging/instances/INSTANCE_ID/partner-activate
Idempotency-Key: poc-canal-01
Content-Type: application/json

{}
```

Guardar **response.id** como channelId: o canal vem em objeto plano, sem wrapper channel. Ativar mensageria não liga bots. Consultar os canais permite recuperar o vínculo pela instanceId.

A POC para o Runner em QR, pending, erro, UNKNOWN ou reconciliationRequired. Escanear e retomar na consulta de estado; ativação/envio exigem CONNECTED da mesma instanceId. run_id preserva as chaves da mesma intenção: só limpar para uma nova POC depois de conciliar a anterior. Pareamento/logout, canal de envio e receptor revogado precisam pertencer à mesma rodada. Não definir IDs gerenciados ou run_id no ambiente/data/local: sobrescrita interrompe a requisição antes do HTTP. O ambiente vazio omite esses nomes.

4.1. Vínculo e identidade da integração direta

Somente um vínculo QR BAILEYS NOVO, criado por administrador autenticado ou chave de controle com chatwoot:manage no contexto aprovado, pode matricular automaticamente sua primeira identidade. Criar a instância genérica não cria esse vínculo e não concede autorização Chatwoot.

A intenção de criação dura até 24 horas. Se a instância já estiver conectada, a conclusão da criação pode matricular a primeira observação válida. Se estiver desconectada, um POST pair administrativo arma uma janela de até dois minutos, limitada também à validade do QR/código e à intenção original. Repetir a mesma chave idempotente não prorroga a janela.

A matrícula confere autoridade atual, tenant, origem HTTPS e revisão do destino, Account, versão da credencial, instância, inbox API, callback e segredo, primeira identidade válida observada e revisão atual. Revogação, mudança de contexto, logout ou divergência de identidade impede consumir a intenção; uma nova ação administrativa pode rearmar somente dentro do prazo permitido e com contexto válido.

GET status e a observação de saúde nunca autorizam nem aprovam identidade. O worker só conclui uma intenção administrativa previamente armada. Vínculos legados, intenção expirada ou uma troca de número conservam a confirmação administrativa explícita da observedRevision atual. Uma identidade já aprovada nunca é substituída automaticamente.

Onboarding SUCCEEDED e integração READY comprovam configuração. CONNECTED comprova sessão observada. A identidade matriculada não comprova callback recebido nem entrega. Consulte saúde e campaign-transport: ready/reason são a capacidade técnica vigente; OPERATIONAL exige ainda evidências recentes de entrada e saída. Bot/Flow não é ativado e a instalação CCLX não é homologada por essa matrícula. O onboarding Meta permanece separado.

A origem/Account/revisão vêm de control/context e do vínculo autenticado, não de organização ou conta arbitrária no corpo. Uma chave de controle pode servir às conexões autorizadas da mesma Account; não se emite uma chave por inbox. O host integra resources/status ao seu catálogo local. A sincronização e a instalação do CCLX precisam de homologação própria. X-JRC-External-Actor é atribuição auditável opcional nas mutações por chave; a reserva de campanha exige ID numérico do administrador remoto e revisões atuais. Esse header não autentica nem concede permissões.

Legado e substituição: consultar status, conferir número mascarado e identityRevision, decidir explicitamente e executar confirm-identity com observedRevision atual e chave idempotente. Não confirmar automaticamente ao receber erro. Flow, transferência e editor usam credenciais e contratos separados; nenhuma coleção deste guia publica automação.

5. Receber e responder

Recebimento por consulta periódica

```
GET /v1/partner-events?after=0&limit=50

# Persistir eventos e o nextCursor retornado.
GET /v1/partner-events?after=CURSOR_RETORNADO&limit=50
```

Cursores são strings decimais. Não converter para Number em JavaScript. Persistir eventId único e cursor no backend; consultar até obter uma página sem avanço e repetir com intervalo controlado. Eventos históricos importados não são tratados como novas mensagens acionáveis.

Responder a uma conversa do Broker

```
POST /v1/messaging/channels/CHANNEL_ID/partner-text
Idempotency-Key: cclx-resposta-123
Content-Type: application/json

{"conversationId":"CONVERSATION_ID_JRC",
 "text":"Mensagem de homologação"}
```

Alternativa: phoneNumber E.164 em vez de conversationId, nunca ambos. Texto: 1 a 4096 caracteres. IDs da conversa local do CCLX não podem ser enviados como UUID da conversa JRC. Usar mapa durável entre os sistemas.

202 significa enfileirado. Consultar mensagens e eventos para SENT/DELIVERED/READ ou falha. A chave é revalidada no despacho: expiração, revogação, exclusão ou remoção do escopo bloqueiam envio pendente. Revogação não recolhe uma requisição já enviada ao provedor.

Evitar loop

Não reenviar mensagens recebidas da JRC de volta ao WhatsApp. Guardar a origem e o ID externo; mensagens espelhadas geram atualização local, não uma nova intenção de saída. Uma intenção do agente usa uma única chave idempotente, inclusive após timeout.

6. Webhooks: assinatura e retomada

```
POST /v1/partner-webhooks
Idempotency-Key: cclx-receptor-01
Content-Type: application/json

{"url":"https://backend.example/hooks/jrc"}
```

Guardar response.endpoint.id e response.signingSecret. O segredo é independente da chave de API. Listagens não devolvem segredos. A URL exige HTTPS público, porta 443, DNS e certificado válidos, sem query, fragmento, usuário ou senha. Redirects e endereços privados são rejeitados. Até cinco endpoints ativos por tenant.

Evento	Significado
message.received	Nova mensagem recebida no Broker.
message.created	Mensagem de saída registrada no Broker, inclusive origem observada.
message.status	Mudança observada no estado da mensagem.
connection.status	Mudança de estado de conexão QR ou Meta.

```
X-JRC-Event-Id: UUID_IMUTAVEL
X-JRC-Timestamp: UNIX_SEGUNDOS
X-JRC-Signature: sha256=HEX_HMAC

HEX_HMAC = HMAC_SHA256(signingSecret,
  timestamp + "." + eventId + "." + exactRawBody)
```

Validar bytes brutos antes de transformar JSON, comparar em tempo constante, conferir header eventId contra o envelope e aceitar timestamp com janela de cinco minutos. Persistir em inbox durável com chave única eventId antes do 2xx. Publicar no Pub/Sub a partir de outbox durável evita perder o evento entre confirmação HTTP e fila.

Entrega ao menos uma vez, possivelmente fora de ordem. Até dez tentativas totais: falhas de rede, 5xx, 408, 425 e 429 permitem retry limitado. Outros códigos, incluindo redirect, são terminais. A JRC não guarda o corpo da resposta do receptor. Revogar uma chave de API não revoga o receptor; revogue ambos ao encerrar a integração.

Retenção do feed: 30 dias. 409 EVENT_CURSOR_EXPIRED informa resetCursor/earliestCursor/lastPrunedCursor: reiniciar e conciliar com o estado das mensagens. Nunca inferir sucesso a partir de feed vazio.

7. Erros, estados e operação segura

Código / estado	Como tratar
400 / 422	Corrigir corpo, parâmetros ou formato conforme schema; não repetir indefinidamente.
401 / 403	Conferir validade, escopo e vínculo da chave. Não trocar tenant ou contornar a autorização.
404	Conferir ID, perfil de integração e disponibilidade; um recurso de outro tenant não fica acessível.
409	Conflito de revisão, identidade, idempotência ou cursor. Consultar e conciliar antes de agir.
429	Respeitar limites e backoff do cliente. Não criar múltiplas intenções para superar o limite.
502 / 503 / timeout	Consultar o estado da intenção existente e usar a mesma Idempotency-Key ao repetir a mesma operação.
UNKNOWN	Resultado incerto. Não gerar nova mensagem/chave para forçar envio; conciliar a operação.
DELIVERED no webhook	O receptor respondeu 2xx; não confundir com mensagem WhatsApp entregue.

Aceite da POC

1. Uma empresa e um número de homologação; chave com prazo e escopos mínimos.
2. Parear, desconectar e reconectar pela interface incorporada.
3. Receber mensagem e registrá-la na caixa correta do CCLX.
4. Responder e observar os estados sem duplicação.
5. Repetir a intenção com a mesma chave e simular queda do receptor.
6. Comprovar isolamento de outro tenant, revogação e retomada por cursor.
7. Confirmar limites de mídia, grupos, Meta e Flow antes de ampliar o escopo.
8. Expandir até cinco números e iniciar o período acordado após aceite do percurso inicial.

Sem acesso ao host do CCLX e sem números de homologação, o pacote pode ser testado por contratos e simulação, mas não é correto certificar a integração externa como concluída.

8. Catálogo de endpoints

As próximas páginas explicam as 34 operações dos perfis públicos. Tipos, campos obrigatórios, limites e respostas são derivados do mesmo OpenAPI do Swagger. Falhas usam application/problem+json com code e requestId. 401 indica credencial inválida/ambígua; 403 permissão ou vínculo; 404 recurso do tenant ou controle desligado; 409 conflito/identidade/reconciliação; 429 limite; 502/503 dependência. Rotas legadas fora do controle podem declarar somente sucesso. Timeout de mutação não prova falha: preservar idempotência, consultar e conciliar antes de outra intenção.

Conexões QR

Conexões da organização. Use X-JRC-API-Key com instances:read ou instances:write. A chave identifica o tenant; não envie organizationId. A modalidade oficial depende do onboarding Meta. Não são endpoints diretos do motor interno.

Consultar contas de provedor disponíveis

```
GET /v1/provider-accounts
```

Lista as contas de provedor da sua organização. Filtre provider=BAILEYS para QR e guarde data[].id como providerAccountId. Uma lista vazia exige provisionamento pela JRC; não use IDs ou chaves do motor interno.

Escopo: instances:read

Parâmetros: limit (query, opcional); cursor (query, opcional); provider (query, opcional)

Respostas declaradas: 200, 400, 401, 403, 404

Listar conexões da empresa

```
GET /v1/instances
```

Retorna uma página das instâncias do tenant. Use limit e o cursor retornado para continuar a listagem.

Escopo: instances:read

Parâmetros: limit (query, opcional); cursor (query, opcional)

Respostas declaradas: 200, 400, 401, 403, 404

Criar uma conexão

```
POST /v1/instances
```

Recebe name, provider e providerAccountId obtido na consulta de provedores. Use BAILEYS para o percurso QR desta POC. A criação é idempotente; não significa que o WhatsApp já está conectado.

Escopo: instances:write

Parâmetros: idempotency-key (header, obrigatório)

Corpo JSON: name *: string, provider *: BAILEYS/META, providerAccountId *: string (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 200, 201, 202, 400, 401, 403, 404, 409, 415, 502

Consultar uma conexão

```
GET /v1/instances/{id}
```

Retorna os dados da instância identificada por um UUID do Broker pertencente à empresa autenticada.

Escopo: instances:read

Parâmetros: id (path, obrigatório)

Respostas declaradas: 200, 400, 401, 403, 404

Solicitar pareamento QR

```
POST /v1/instances/{id}/connect
```

Recebe {} ou pairingHint opcional. Inicia ou observa a conexão; o resultado pode ser imediato ou uma operação pendente. Confira o schema de resposta e consulte status. Exiba o QR apenas ao operador autorizado.

Escopo: instances:write

Parâmetros: id (path, obrigatório); idempotency-key (header, obrigatório)

Corpo JSON: pairingHint: string (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 200, 202, 400, 401, 403, 404, 409, 415, 502

Consultar estado de conexão

```
GET /v1/instances/{id}/status
```

Observa o estado atual do provedor. Um QR exibido ou uma requisição aceita não comprova pareamento concluído.

Escopo: instances:read

Parâmetros: id (path, obrigatório)

Respostas declaradas: 200, 400, 401, 403, 404, 502

Desconectar uma instância

```
POST /v1/instances/{id}/disconnect
```

Recebe {}. Solicita desconexão e pode devolver operação pendente. Interrompe o transporte dessa instância; use apenas no número de homologação ou com autorização operacional.

Escopo: instances:write

Parâmetros: id (path, obrigatório); idempotency-key (header, obrigatório)

Corpo JSON: {} (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 200, 202, 400, 401, 403, 404, 409, 415, 502

Mensageria por chave

Tenant inferido da chave JRC. Use messaging:read para consultas e messaging:send para texto QR; ativar canal exige também instances:write. Não exige URL, Account ID ou credencial Chatwoot. Envio atual deste perfil: texto individual WhatsApp QR; inclui download privado de anexo, mas não envio de mídia, grupos, templates Meta nem Flows. 202 confirma fila, não entrega; acompanhe mensagens/eventos. Nunca repita uma operação UNKNOWN com outra chave de idempotência.

Ativar mensageria para uma instância QR

```
POST /v1/messaging/instances/{id}/partner-activate
```

Recebe {}. Cria ou reutiliza o canal canônico da instância QR do tenant. Guarde response.id como channelId; a resposta é um objeto plano. Não publica nem ativa bot.

Escopo: instances:write + messaging:send

Parâmetros: id (path, obrigatório); idempotency-key (header, obrigatório)

Corpo JSON: {} (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 502, 503

Listar canais de mensageria

```
GET /v1/messaging/channels
```

Retorna os canais visíveis ao tenant. Use id para conversas/envio; instanceId identifica a instância associada quando disponível.

Escopo: messaging:read

Respostas declaradas: 200

Listar conversas de um canal

```
GET /v1/messaging/channels/{id}/conversations
```

Consulta as conversas registradas no Broker para o canal. Não representa todo o histórico existente no aplicativo WhatsApp; depende dos eventos recebidos e do histórico efetivamente importado.

Escopo: messaging:read

Parâmetros: id (path, obrigatório)

Respostas declaradas: 200

Consultar uma conversa

```
GET /v1/messaging/conversations/{id}
```

Retorna os dados da conversa canônica. IDs de conversa do Broker não são IDs de conversa do Chatwoot/CCLX; mantenha um mapa no adaptador.

Escopo: messaging:read

Parâmetros: id (path, obrigatório)

Respostas declaradas: 200

Consultar mensagens e estados

```
GET /v1/messaging/conversations/{id}/messages
```

Retorna as mensagens registradas e seus estados observados. Pode incluir metadados de mídia; use media.id na rota de download privado. Este perfil ainda não disponibiliza envio de mídia.

Escopo: messaging:read

Parâmetros: id (path, obrigatório)

Respostas declaradas: 200

Baixar anexo privado registrado no Broker

```
GET /v1/messaging/media/{id}
```

Devolve bytes do anexo com seu MIME e Content-Disposition attachment. Use media.id de uma mensagem. A chave e o tenant são revalidados antes de liberar os bytes. Não é URL pública nem envio de arquivo; o backend consumidor deve respeitar permissões locais e limites.

Escopo: messaging:read

Parâmetros: id (path, obrigatório)

Respostas declaradas: 200

Enfileirar texto individual pelo QR

```
POST /v1/messaging/channels/{id}/partner-text
```

Recebe conversationId OU phoneNumber E.164, e text com 1 a 4096 caracteres. Usa o envio humano canônico e revalida a autoridade da chave no despacho. HTTP 202 confirma fila, não entrega. Suporte atual: texto individual QR; não mídia, grupos ou templates Meta.

Escopo: messaging:send

Parâmetros: id (path, obrigatório); idempotency-key (header, obrigatório)

Corpo JSON: conversationId *: string, text *: string OU phoneNumber *: string, text *: string (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 202, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503

Eventos e webhooks

Exige webhooks:manage. Endpoint do consumidor pode ser cadastrado depois da emissão da chave. Entrega ao menos uma vez com HMAC-SHA256 e tentativas limitadas; deduplique eventId. Ordem de entrega não é garantida; use cursor e não reverta estados mais recentes. Polling retém 30 dias e retorna 409 quando o cursor expira. O segredo de assinatura é separado da chave de API. Revogar uma chave de API não revoga automaticamente endpoints: revogue-os explicitamente.

Listar endpoints receptores

```
GET /v1/partner-webhooks
```

Retorna os endpoints cadastrados sem seus segredos de assinatura. API key e segredo HMAC são credenciais distintas.

Escopo: webhooks:manage

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503

Cadastrar receptor HTTPS assinado

```
POST /v1/partner-webhooks
```

Recebe url HTTPS pública, porta 443, sem query/fragmento ou credenciais. Máximo de cinco endpoints ativos por empresa. Retorna endpoint, signingSecret e replayed; guarde o segredo no backend. Repetição com a mesma chave idempotente e URL recupera o cadastro.

Escopo: webhooks:manage

Parâmetros: idempotency-key (header, obrigatório)

Corpo JSON: url *: string (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 201, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503

Revogar receptor e cancelar entregas pendentes

```
DELETE /v1/partner-webhooks/{id}
```

Revoga o endpoint, remove seu segredo e cancela trabalho pendente. Uma requisição já despachada pode chegar. Revogar a chave de API não revoga esse endpoint automaticamente.

Escopo: webhooks:manage

Parâmetros: id (path, obrigatório)

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503

Consultar eventos por cursor

```
GET /v1/partner-events
```

Use after=0 na primeira leitura e depois o nextCursor retornado. after e cursores são strings decimais; limit de 1 a 100. Retenção de 30 dias. Cursor expirado retorna 409 EVENT_CURSOR_EXPIRED; reinicie a leitura e concilie com mensagens.

Escopo: webhooks:manage

Parâmetros: after (query, opcional); limit (query, opcional)

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503

Consultar diagnóstico de entregas HTTP

```
GET /v1/partner-webhook-deliveries
```

Lista até 100 entregas com estado e tentativas limitadas. DELIVERED aqui significa que o receptor confirmou HTTP 2xx; não significa entrega de mensagem WhatsApp.

Escopo: webhooks:manage

Parâmetros: limit (query, opcional)

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503

Controle Chatwoot opcional

Perfil opcional para uma Account Chatwoot vinculada ao Broker. Origem HTTPS aprovada, Account e revisão pertencem ao vínculo da credencial; não envie organizationId/accountId para escolher outra empresa. Exige chave de controle específica chatwoot:read/manage/pair/disconnect; a chave genérica de instâncias não autoriza esse perfil. Novos vínculos QR criados por administrador/manage podem matricular a primeira identidade automaticamente sob intenção finita e verificação do provider, contexto e inbox. GET não aprova identidade; legado e troca de número conservam confirmação administrativa. Mensagens dos agentes retornam pelo webhook da caixa. Flow possui credencial e contrato separados; este perfil não publica bots nem comprova homologação CCLX.

Consultar contexto de controle Chatwoot

```
GET /v1/integrations/chatwoot/control/context
```

Perfil opcional vinculado a uma Account Chatwoot remota. Consulta o contexto autorizado dessa integração; não usa a chave genérica de mensageria.

Escopo: chatwoot:read

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Listar recursos para vínculo Chatwoot

```
GET /v1/integrations/chatwoot/control/resources
```

Obtém recursos autorizados para o assistente de conexão, incluindo instâncias e escolhas locais/remotas disponíveis.

Escopo: chatwoot:manage

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Consultar operações de vínculo Chatwoot

```
GET /v1/integrations/chatwoot/control/onboarding
```

Lista até 50 operações persistidas no contexto autorizado da Account, incluindo estado, etapa e IDs disponíveis. Não inicia configuração nem matricula identidade. As escolhas de provedores/instâncias vêm de resources.

Escopo: chatwoot:manage

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Iniciar vínculo de caixa e conexão

```
POST /v1/integrations/chatwoot/control/onboarding
```

Recebe name, source NEW/EXISTING, agentIds, inboxId opcional e replaceExistingWebhook, conforme schema. Cria um vínculo novo com intenção administrativa finita para a primeira identidade QR; não altera automaticamente vínculos legados. Retorna 202 com operationId; consulte até conclusão. SUCCEEDED configura a caixa, mas não comprova pareamento ou entrega. Não substituir webhook existente sem decisão explícita.

Escopo: chatwoot:manage

Parâmetros: idempotency-key (header, obrigatório); X-JRC-External-Actor (header, opcional)

Corpo JSON: agentIds *: array, inboxId: integer, name *: string, replaceExistingWebhook *: boolean, source *: objeto (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 202, 400, 401, 403, 404, 409, 413, 415, 502, 503

Consultar operação de vínculo

```
GET /v1/integrations/chatwoot/control/onboarding/{operationId}
```

Consulta os passos e o resultado da operação de onboarding existente, sem repetir a criação.

Escopo: chatwoot:read

Parâmetros: operationId (path, obrigatório)

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Recuperar operação de vínculo

```
POST /v1/integrations/chatwoot/control/onboarding/{operationId}/recover
```

Recebe action RETRY, RECONCILE ou CANCEL conforme estado permitido. Resultados incertos exigem conciliação; não force criação duplicada.

Escopo: chatwoot:manage

Parâmetros: operationId (path, obrigatório); idempotency-key (header, obrigatório); X-JRC-External-Actor (header, opcional)

Corpo JSON: action *: RETRY/RECONCILE/CANCEL (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 202, 400, 401, 403, 404, 409, 413, 415, 502, 503

Consultar saúde da conexão integrada

```
GET /v1/integrations/chatwoot/control/connections/{integrationId}/status
```

Observa sessão, identityApproved, identityStatus, identityRevision, número mascarado, callback, evidências e allowedActions. GET nunca aprova identidade nem gera QR. READY é configuração; CONNECTED é sessão; OPERATIONAL exige identidade e evidências recentes na revisão atual. Vínculo cadastrado não comprova entrega.

Escopo: chatwoot:read

Parâmetros: integrationId (path, obrigatório)

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Solicitar QR da conexão integrada

```
POST /v1/integrations/chatwoot/control/connections/{integrationId}/pair
```

Recebe {} e inicia/reutiliza pareamento autorizado. Em vínculo novo elegível, um administrador/manage arma a matrícula inicial finita. Reconectar identidade já aprovada pode usar delegação pair vigente; divergência retorna IDENTITY_CONFIRMATION_REQUIRED e exige decisão administrativa. Resposta pode ser imediata ou 202 pendente; consulte pair-operations e status. Não repetir uma operação incerta com nova chave.

Escopo: chatwoot:pair; primeira identidade exige também chatwoot:manage

Parâmetros: integrationId (path, obrigatório); idempotency-key (header, obrigatório); X-JRC-External-Actor (header, opcional)

Corpo JSON: {} (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 200, 202, 400, 401, 403, 404, 409, 413, 415, 502, 503

Consultar operação de pareamento

```
GET /v1/integrations/chatwoot/control/connections/{integrationId}/pair-operations/{operationId}
```

Obtém progresso durável sem iniciar outra operação nem aprovar identidade. Um QR/código efêmero pode ser liberado uma vez, somente com autoridade e contexto revalidados; expiração, falha de cache ou leitura anterior podem deixar action nula. O identificador pertence à integração e ao tenant; UNKNOWN exige conciliação, não novo pair.

Escopo: chatwoot:pair; identidade ainda não aprovada exige também chatwoot:manage

Parâmetros: integrationId (path, obrigatório); operationId (path, obrigatório)

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Confirmar identidade observada do QR

```
POST /v1/integrations/chatwoot/control/connections/{integrationId}/confirm-identity
```

Ação administrativa explícita para legado sem matrícula elegível ou troca de número. Recebe observedRevision e confirma exatamente a identidade observada atual, sem aceitar revisão desatualizada. Não é um passo automático após toda criação; conferir número e impacto antes de executar.

Escopo: chatwoot:manage

Parâmetros: integrationId (path, obrigatório); idempotency-key (header, obrigatório); X-JRC-External-Actor (header, opcional)

Corpo JSON: observedRevision *: integer (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Desconectar número da integração

```
POST /v1/integrations/chatwoot/control/connections/{integrationId}/disconnect
```

Recebe {} e solicita a desconexão da instância vinculada. Pode gerar operação pendente; consulte o resultado.

Escopo: chatwoot:disconnect

Parâmetros: integrationId (path, obrigatório); idempotency-key (header, obrigatório); X-JRC-External-Actor (header, opcional)

Corpo JSON: {} (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 200, 202, 400, 401, 403, 404, 409, 413, 415, 502, 503

Definir agentes da caixa integrada

```
PUT /v1/integrations/chatwoot/control/connections/{integrationId}/agents
```

Recebe agentIds da Account vinculada. Aplica o conjunto permitido de agentes; não concede acesso a outra empresa ou a outra Account.

Escopo: chatwoot:manage

Parâmetros: integrationId (path, obrigatório); idempotency-key (header, obrigatório); X-JRC-External-Actor (header, opcional)

Corpo JSON: agentIds *: array (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Consultar capacidade real do remetente para campanhas

```
GET /v1/integrations/chatwoot/control/connections/{integrationId}/campaign-transport
```

Retorna protocolo chatwoot-campaigns/v1, vínculo, revisões, identidade, ready e reason. Não concede acesso nem muda executor. A reserva de campanhas exige hook compatível no host; ready de transporte não certifica instalação do CCLX.

Escopo: chatwoot:read

Parâmetros: integrationId (path, obrigatório)

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Reservar intenção canônica de campanha

```
POST /v1/integrations/chatwoot/control/connections/{integrationId}/campaign-intents
```

Exige chave de controle vinculada e X-JRC-External-Actor. O host grava a mensagem canônica, reserva a intenção e só depois envia o único callback assinado. O Broker confere ator, conta, caixa, conteúdo e revisões. Não chamar diretamente o motor nem duplicar o envio pelo caminho genérico.

Escopo: chatwoot:manage

Parâmetros: integrationId (path, obrigatório); idempotency-key (header, obrigatório); x-jrc-external-actor (header, obrigatório)

Corpo JSON: campaignId *: string, conversationId *: integer, destinationRevision *: integer, identityRevision *: integer, kind *: text/image/document/video/audio, messageId *: integer, ownerRevision *: integer, text *: string (* obrigatório). Ver schemas completos no Swagger.

Respostas declaradas: 200, 201, 400, 401, 403, 404, 409, 413, 415, 502, 503

Consultar recibo agregado da campanha

```
GET /v1/integrations/chatwoot/control/connections/{integrationId}/conversations/{conversationId}/campaign-messages/{messageId}
```

Usa IDs numéricos da conversa e mensagem da central vinculada, não UUIDs de conversa JRC. Agrega os anexos conservadoramente; queued não significa sent, e unknown exige conciliação.

Escopo: chatwoot:read

Parâmetros: integrationId (path, obrigatório); conversationId (path, obrigatório); messageId (path, obrigatório)

Respostas declaradas: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

9. Capacidades além da POC inicial

A meta é um Broker completo, mas capacidade do motor, contrato JRC e homologação são provas diferentes. Não tratar uma rota existente ou um botão habilitado como prova de entrega. QR e Meta também têm capacidades distintas.

Recurso	Situação e próximo critério
Texto QR	Contrato por chave documentado. Provar ida/volta no número de homologação e estados reais.
Mídia	Há transporte de imagem/áudio/vídeo/documento/figurinha no Broker. O perfil genérico de chave já permite download privado pelo ID da mensagem; ainda precisa de upload/envio, cotas e homologação específica.
Campanhas	São envios individuais agendados. A ponte JRC exige contrato canônico de capacidade, intenção e recibo da revisão correspondente; não basta liberar uma checkbox.
Grupos e histórico	Há caminhos próprios no Broker, com seleção e revisões. Não presumir catálogo de grupos completo, administração de participantes ou cópia integral do WhatsApp pelo perfil genérico.
Flow / URA	Usar primeiro o editor no console Broker, com um executor por canal. Editor incorporado, catálogo remoto, transferência e retomada precisam de adaptação ao host CCLX. A chave QR/texto não autoriza gestão do Flow, e os quatro eventos genéricos não executam atribuição local. A POC genérica não cobre esse controle.
Chamadas	Na referência Evolution fixada, offerCall tem a chamada real comentada e devolve um ID de exemplo. Voz exige transporte efetivo e validado; esse stub não comprova ligação.
Outros tipos	Localização, cartão de contato, reação, enquete, lista e botões têm funções na referência Evolution. Cada tipo precisa de contrato, armazenamento/projeção, despacho e prova próprios no Broker.
Monitoramento	Usar saúde, fila e diagnóstico. UNKNOWN exige conciliação. Eventos e Pub/Sub não autorizam repetir efeitos de envio.

Modelo CCLX: uma organização Casa do Construtor CCLX, com uma instância por número. Usuários próprios usam o console Broker; o backend CCLX usa a chave do tenant. O CCLX relaciona agente/Account/caixa a instanceId e channelId; um agente pode atender várias caixas e um número pode ser compartilhado por agentes autorizados.

10. Referências e verificação de versão

Fonte contratual: docs/api/openapi.json e perfis públicos gerados em scripts/docs/public-api.mjs. A geração falha se uma rota selecionada desaparecer; a documentação não cria endpoints fictícios.

Operações neste catálogo: 34. Código dos contratos canônicos SHA-256:
0faaa45e31d9bee9548527370e9a0128cf77aad07305e6dc64fa6673592c4783

Este manual deve acompanhar a mesma versão do Swagger e das imagens API/Web. A publicação no GHCR não atualiza o Dokploy automaticamente. O operador precisa aplicar as migrações exigidas pela revisão, atualizar todos os workers e conferir prontidão e logs.

Documentação oficial de referência para o adaptador Chatwoot:
Como criar uma caixa do tipo API
Webhooks do Chatwoot

Essas referências descrevem o produto de origem e não certificam o fork/versão do CCLX. A versão exata e as adaptações devem ser verificadas no ambiente de homologação.